

Systeme d'invitation de compte — Guide Backend

Projet : Holy Fork **Repo backend :** holly_pi (Django REST Framework) **Date :** 19 juillet 2026 **Auteur :** Equipe frontend

Introduction

Ce document décrit les modifications nécessaires côté backend pour que le système d'invitation d'employés fonctionne correctement de bout en bout.

Le contexte : Quand un gérant crée un nouvel employé dans le dashboard, il peut cocher "Créer un accès dashboard" pour lui générer un compte utilisateur (email + mot de passe temporaire). Côté frontend, tout est en place : formulaire, validation, affichage des credentials. Côté backend, l'endpoint existe (POST /api/settings/users/) mais il manque des verrous de sécurité et certaines données sont perdues en route.

Ce que fait le frontend aujourd'hui :

1. Le gérant remplit le formulaire (nom, prénom, email, type d'employé, salaire, date d'embauche, téléphone, établissement)
2. Le frontend génère un mot de passe aléatoire de 12 caractères et un PIN de 4 chiffres
3. Il envoie tout ça en un seul POST vers /api/settings/users/
4. En cas de succès, il affiche les credentials au gérant pour qu'il les transmette à l'employé

Les problèmes qu'on a identifiés (par ordre de gravité) :

#	Probleme	Gravite
1	N'importe quel utilisateur authentifié peut créer des comptes (pas de vérification de permission)	Critique
2	Le salaire, la date d'embauche et le téléphone envoyés par le frontend sont ignorés par le backend	Bloquant
3	La liste des users n'est pas filtrée par restaurant (expose tous les users)	Important
4	Impossible de désactiver un compte employé	Important
5	L'email de l'employé n'est pas retourné dans GET /api/employees/	Mineur
6	Pas d'envoi d'email d'invitation	Amélioration
7	Pas de changement / reset de mot de passe	Amélioration

Le document est organisé en 3 niveaux de priorité. Chaque section explique le problème, pourquoi c'est un problème, et propose une implémentation possible. Le code est donné à titre indicatif — libre à toi de l'adapter selon tes conventions.

P0 — Corrections critiques

Ces deux points doivent être traités avant toute mise en production. Sans eux, le système présente une faille de sécurité majeure et des données sont perdues.

1. Sécuriser la création de compte (permission MANAGE_STAFF)

Où : apps/settings/views.py, classe UsersSettingsView (ligne 86)

Le problème en clair : Aujourd'hui, la vue n'a que `permission_classes = [IsAuthenticated]`. Ça veut dire que n'importe quel user connecté — un serveur, un stagiaire, n'importe qui — peut appeler ce endpoint et créer un compte avec le rôle qu'il veut. Un serveur pourrait se créer un compte "Admin Etablissement" et avoir accès à tout.

Côté frontend, le bouton "Créer" est masqué si l'utilisateur n'a pas la permission `manage_staff`. Mais un curl direct bypass complètement cette protection.

Ce qu'il faut faire : Ajouter la permission `MANAGE_STAFF` sur la méthode POST. Le GET peut rester accessible à tout user authentifié (il sert à afficher la liste des collègues).

Proposition d'implémentation :

```
# apps/settings/views.py

from apps.staff.permissions import HasPermission
from apps.staff.employee_roles import Permission

class UsersSettingsView(APIView):
    """
    GET /api/settings/users?restaurant_id=X — liste des utilisateurs
    POST /api/settings/users — création de compte (gerant uniquement)
    """
    permission_classes = [IsAuthenticated]
```

```

required_permission = Permission.MANAGE_STAFF

def get_permissions(self):
    """
    Le POST requiert MANAGE_STAFF en plus de l'authentification.
    Le GET reste ouvert a tout user authentifie.
    """
    if self.request.method == 'POST':
        return [IsAuthenticated(), HasPermission()]
    return [IsAuthenticated()]

def get(self, request):
    # (voir section 3 pour le filtrage par restaurant)
    restaurant_id = request.query_params.get('restaurant_id')
    if restaurant_id:
        from apps.staff.models import RestaurantEmploye
        employe_ids = RestaurantEmploye.objects.filter(
            restaurant_id=restaurant_id
        ).values_list('employe__user_id', flat=True)
        users = HollyUser.objects.filter(id__in=employe_ids)
    else:
        users = HollyUser.objects.all()
    serializer = UserProfileSerializer(users, many=True)
    return Response(serializer.data)

def post(self, request):
    serializer = UserRegistrationSerializer(
        data=request.data,
        context={'request': request, 'is_admin_invite': True}
    )
    if serializer.is_valid():
        user = serializer.save()
        return Response(UserProfileSerializer(user).data, status=201)
    return Response(serializer.errors, status=400)

```

Comment verifier que ca marche :

- Se connecter avec un compte "Serveur" → POST doit retourner 403
- Se connecter avec un compte "Admin Etablissement" ou "Directeur" → POST doit retourner 201

2. Accepter les donnees completes de l'employe (salaire, date, telephone)

Ou : apps/authentication/serializers.py, classe UserRegistrationSerializer (ligne 22)

Le probleme en clair : Le frontend envoie salary, hire_date et phone_number dans le body du POST, mais ces champs ne sont pas declares dans le serializer. Django REST Framework les ignore silencieusement. Resultat : l'employe est cree avec un salaire a 0, la date du jour, et pas de telephone — meme si le gerant a tout rempli dans le formulaire.

En plus, restaurant_id est declare required=True. Ca empeche de creer un employe sans l'assigner immediatement a un restaurant (cas ou le gerant veut creer le compte d'abord et assigner plus tard).

Ce qu'il faut faire :

1. Ajouter salary, hire_date, phone_number comme champs optionnels du serializer
2. Les utiliser dans create() pour remplir le modele Employe
3. Rendre restaurant_id optionnel (mais garder l'obligation pour l'auto-inscription via /api/auth/register)

Proposition d'implementation :

```

# apps/authentication/serializers.py

from decimal import Decimal
from django.utils import timezone

class UserRegistrationSerializer(serializers.ModelSerializer):
    """
    Serializer pour la creation d'un utilisateur + employe.
    Utilise par deux endpoints :
    - /api/auth/register (auto-inscription → restaurant_id obligatoire)
    - /api/settings/users/ (invitation admin → restaurant_id optionnel)
    """
    email = serializers.EmailField(
        required=True,
        validators=[UniqueValidator(queryset=User.objects.all())]
    )
    password = serializers.CharField(
        write_only=True, required=True,
        validators=[validate_password]
    )
    password2 = serializers.CharField(write_only=True, required=True)

```

```

# Identite employe
employee_last_name = serializers.CharField(required=True, max_length=100)
employee_first_name = serializers.CharField(required=True, max_length=100)
pin_code = serializers.CharField(
    required=True, max_length=4, min_length=4,
    validators=[RegexValidator(r'^\d{4}$', 'Le code PIN doit contenir exactement 4 chiffres.')]
)
type_employe_id = serializers.PrimaryKeyRelatedField(
    queryset=TypeEmploye.objects.all(), required=True, write_only=True
)

# Restaurant - CHANGEMENT : plus obligatoire
restaurant_id = serializers.PrimaryKeyRelatedField(
    queryset=Restaurant.objects.all(),
    required=False,
    allow_null=True,
    write_only=True
)

# NOUVEAUX CHAMPS : donnees employe completes
salary = serializers.DecimalField(
    max_digits=10, decimal_places=2,
    required=False, write_only=True, default=Decimal('0.00')
)
hire_date = serializers.DateField(required=False, write_only=True)
phone_number = serializers.CharField(
    max_length=20, required=False, write_only=True, allow_blank=True
)

class Meta:
    model = User
    fields = (
        'username', 'email', 'password', 'password2',
        'first_name', 'last_name',
        'employee_first_name', 'employee_last_name',
        'pin_code', 'type_employe_id', 'restaurant_id',
        'salary', 'hire_date', 'phone_number',
    )

def validate(self, attrs):
    if attrs['password'] != attrs['password2']:
        raise serializers.ValidationError({
            "password": "Les mots de passe ne correspondent pas."
        })

    # Unicité du PIN dans le restaurant (seulement si restaurant fourni)
    pin_code = attrs.get('pin_code')
    restaurant = attrs.get('restaurant_id')

    if restaurant and pin_code:
        existing = RestaurantEmploye.objects.filter(
            restaurant=restaurant
        ).select_related('employe')
        for re in existing:
            if re.employe.pin_code and re.employe.pin_code == pin_code:
                raise serializers.ValidationError({
                    "pin_code": "Ce PIN est déjà utilisé dans ce restaurant."
                })

    # L'auto-inscription (sans contexte admin) exige toujours un restaurant
    if not restaurant and not self.context.get('is_admin_invite'):
        raise serializers.ValidationError({
            "restaurant_id": "Ce champ est obligatoire."
        })

    return attrs

def create(self, validated_data):
    # Extraire les champs supplémentaires
    validated_data.pop('password2', None)
    employee_last_name = validated_data.pop('employee_last_name')
    employee_first_name = validated_data.pop('employee_first_name')
    pin_code = validated_data.pop('pin_code')
    type_employe = validated_data.pop('type_employe_id')
    restaurant = validated_data.pop('restaurant_id', None)
    salary = validated_data.pop('salary', Decimal('0.00'))
    hire_date = validated_data.pop('hire_date', None)
    phone_number = validated_data.pop('phone_number', '')

    # 1. Créer le user Django
    user = User.objects.create_user(
        username=validated_data['username'],
        email=validated_data['email'],
        password=validated_data['password'],
        first_name=validated_data.get('first_name', ''),
        last_name=validated_data.get('last_name', '')
    )

    # 2. Créer l'employe avec TOUTES les donnees

```

```

    employe = Employee.objects.create(
        user=user,
        nom=employee_last_name,
        prenom=employee_first_name,
        pin_code=pin_code,
        type_employe=type_employe,
        salaire=salary,
        date_embauche=hire_date or timezone.localtime(),
        numero_telephone=phone_number or '',
    )

    # 3. Associer au restaurant si fourni
    if restaurant:
        RestaurantEmployee.objects.create(
            restaurant=restaurant,
            employe=employe
        )

    return user

```

Comment verifier que ca marche :

1. Depuis le frontend, inviter un employe avec salaire=2500, date=2026-01-15, tel=+33612345678
2. Verifier en base que l'Employe a bien ces valeurs (pas 0.00, pas la date du jour, pas vide)
3. Tester /api/auth/register sans restaurant_id → doit toujours retourner 400

P1 — Fonctionnalités manquantes

Ces points ne sont pas bloquants pour un premier test, mais necessaires pour une utilisation réelle.

3. Filtrer la liste des users par restaurant

Ou : apps/settings/views.py, methode get de UsersSettingsView

Le probleme en clair : Aujourd'hui le GET retourne TOUS les users du systeme, peu importe le restaurant_id passe en parametre. Si on a 3 restaurants avec 30 employes chacun, un gerant du restaurant A voit aussi les employes de B et C.

Ce qu'il faut faire : Filtrer via la table de liaison RestaurantEmployee. C'est deja inclus dans la proposition de code de la section 1 (le if restaurant_id: qui filtre).

Pour aller plus loin, on pourrait aussi verifier que l'utilisateur a bien acces au restaurant demande :

```

# Proposition d'amelioration dans le GET
from apps.staff.permissions_utils import PermissionChecker

def get(self, request):
    restaurant_id = request.query_params.get('restaurant_id')
    checker = PermissionChecker(user=request.user)

    if restaurant_id:
        # Verifier que le demandeur a acces a ce restaurant
        if not checker.has_access_to_restaurant(int(restaurant_id)):
            return Response({'error': 'Acces refuse'}, status=403)
        employe_ids = RestaurantEmployee.objects.filter(
            restaurant_id=restaurant_id
        ).values_list('employe__user_id', flat=True)
        users = HollyUser.objects.filter(id__in=employe_ids)
    else:
        # Sans filtre : seulement les users des restaurants accessibles
        from apps.staff.employee_roles import EmployeeRole
        if checker.role == EmployeeRole.SUPER_ADMIN GROUPE:
            users = HollyUser.objects.all()
        else:
            my_restaurants = RestaurantEmployee.objects.filter(
                employe__user=request.user
            ).values_list('restaurant_id', flat=True)
            employe_ids = RestaurantEmployee.objects.filter(
                restaurant_id__in=my_restaurants
            ).values_list('employe__user_id', flat=True)
            users = HollyUser.objects.filter(id__in=employe_ids)

    return Response(UserProfileSerializer(users, many=True).data)

```

4. Permettre la desactivation d'un compte

Ou : apps/authentication/serializers.py + apps/settings/views.py

Le probleme en clair : Quand un employe quitte le restaurant, le gerant veut desactiver son acces sans supprimer ses donnees (historique, planning passe, etc.). Aujourd'hui `is_active` est en lecture seule dans `UserProfileSerializer` — impossible de le modifier via l'API.

Ce qu'il faut faire : Creer un serializer dedie aux admins avec `is_active` en ecriture, et l'utiliser dans le PATCH.

Proposition :

```
# apps/authentication/serializers.py – nouveau serializer

class AdminUserUpdateSerializer(serializers.ModelSerializer):
    """
    Mise a jour admin d'un compte. Permet de modifier :
    - email, first_name, last_name (classique)
    - is_active (pour desactiver/reactiver un compte)
    """
    class Meta:
        model = User
        fields = ('id', 'username', 'email', 'first_name', 'last_name', 'is_active')
        read_only_fields = ('id', 'username')

# apps/settings/views.py – modifier UserSettingsDetailView

class UserSettingsDetailView(APIView):
    permission_classes = [IsAuthenticated]
    required_permission = Permission.MANAGE_STAFF

    def get_permissions(self):
        if self.request.method == 'PATCH':
            return [IsAuthenticated(), HasPermission()]
        return [IsAuthenticated()]

    def get(self, request, user_id):
        try:
            user = HollyUser.objects.get(id=user_id)
            return Response(UserProfileSerializer(user).data)
        except HollyUser.DoesNotExist:
            return Response({'error': 'Utilisateur introuvable'}, status=404)

    def patch(self, request, user_id):
        try:
            user = HollyUser.objects.get(id=user_id)
            serializer = AdminUserUpdateSerializer(user, data=request.data, partial=True)
            if serializer.is_valid():
                serializer.save()
                return Response(UserProfileSerializer(user).data)
            return Response(serializer.errors, status=400)
        except HollyUser.DoesNotExist:
            return Response({'error': 'Utilisateur introuvable'}, status=404)
```

Comment verifier :

- PATCH avec `{"is_active": false}` → le user ne peut plus se connecter
- PATCH avec `{"is_active": true}` → le user peut se reconnecter
- Un serveur qui tente le PATCH → 403

5. Ajouter l'email dans la reponse des employes

Ou : `apps/staff/serializers.py`, classe `EmployeSerializer`

Le probleme en clair : La reponse de GET `/api/employes/` contient `user_id` mais pas l'email du user lie. Le frontend ne peut donc pas afficher l'email d'un employe qui a un compte.

Ce qu'il faut faire : Ajouter un champ email en lecture seule qui va chercher l'email du user lie.

Proposition :

```
# apps/staff/serializers.py – ajouter dans EmployeSerializer

class EmployeSerializer(serializers.HyperlinkedModelSerializer):
    # ... champs existants ...

    # Nouveau champ
    email = serializers.SerializerMethodField()

    class Meta:
        model = Employe
        fields = [
            'id', 'user_id', 'last_name', 'first_name',
```

```

        'type_employe_id', 'type_employe_name',
        'salary', 'hire_date', 'phone_number',
        'email', # ajouter ici
    ]

def get_email(self, obj):
    """Retourne l'email du user associe, ou None si pas de compte."""
    return obj.user.email if obj.user else None

```

6. Empêcher un employé de se promouvoir

Ou : apps/staff/serializers.py, methode update de EmployeeSerializer

Le probleme en clair : Le PATCH sur un employé accepte un changement de type_employe_id sans verifier la hierarchie. En theorie, un Manager Salle pourrait modifier son propre type pour devenir Admin Etablissement.

Ce qu'il faut faire : Avant d'appliquer un changement de role, verifier que le demandeur a un role superieur au nouveau role assigne. La fonction is_higher_role() existe deja dans employee_roles.py.

Proposition :

```

# apps/staff/serializers.py – dans la methode update de EmployeeSerializer

from apps.staff.employee_roles import EmployeeRole, is_higher_role
from apps.staff.permissions_utils import PermissionChecker

def update(self, instance, validated_data):
    # Si on change le role, verifier la hierarchie
    new_type = validated_data.get('type_employe')
    if new_type and new_type != instance.type_employe:
        request = self.context.get('request')
        if request and request.user:
            checker = PermissionChecker(user=request.user)
            try:
                new_role = EmployeeRole(new_type.nom_type)
                if not is_higher_role(checker.role, new_role):
                    raise serializers.ValidationError({
                        "type_employe_id": "Vous ne pouvez pas assigner un role superieur ou egal au votre"
                    })
            except ValueError:
                pass # Role pas dans l'enum, on laisse passer

    # Appliquer les modifications normalement
    # ... (reste du code existant inchangé)

```

P2 — Améliorations UX

Ces points ne sont pas urgents mais améliorent beaucoup l'expérience pour les employés invités.

7. Changement de mot de passe (user connecté)

Ou : apps/authentication/views.py + apps/authentication/urls.py

Pourquoi : Les employés invités reçoivent un mot de passe temporaire généré. Ils doivent pouvoir le changer une fois connectés.

Endpoint : POST /api/auth/change-password/

Body attendu :

```

{
  "old_password": "MotDePasseActuel123",
  "new_password": "NouveauMotDePasse456!",
  "new_password2": "NouveauMotDePasse456!"
}

```

Reponses :

- 200: {"message": "Mot de passe modifié avec succès."}
- 400: {"old_password": ["Mot de passe actuel incorrect."]}
- 400: {"new_password": ["Les mots de passe ne correspondent pas."]}

Proposition d'implémentation :

```
# apps/authentication/views.py

class ChangePasswordView(APIView):
    """Permet a un user connecte de changer son mot de passe."""
    permission_classes = [IsAuthenticated]

    def post(self, request):
        old_password = request.data.get('old_password')
        new_password = request.data.get('new_password')
        new_password2 = request.data.get('new_password2')

        if not request.user.check_password(old_password):
            return Response(
                {'old_password': ['Mot de passe actuel incorrect.']}, status=400
            )

        if new_password != new_password2:
            return Response(
                {'new_password': ['Les mots de passe ne correspondent pas.']}, status=400
            )

        try:
            validate_password(new_password, request.user)
        except Exception as e:
            return Response({'new_password': list(e.messages)}, status=400)

        request.user.set_password(new_password)
        request.user.save()
        return Response({'message': 'Mot de passe modifie avec succes.'})
```

8. Reset de mot de passe (oublie)

Ou : apps/authentication/views.py + apps/authentication/urls.py

Pourquoi : Un employe qui a oublie son mot de passe doit pouvoir en demander un nouveau par email, sans solliciter le gerant.

Deux endpoints necessaires :

8a. Demande de reset : POST /api/auth/forgot-password/

- Body : {"email": "employe@restaurant.fr"}
- Reponse (toujours 200 pour ne pas reveler si le compte existe) : {"message": "Si un compte existe avec cet email, un lien a ete envoye."}
- En coulisse : genere un token, envoie un email avec un lien vers le frontend

8b. Confirmation du reset : POST /api/auth/reset-password/

- Body : {"uid": "...", "token": "...", "new_password": "...", "new_password2": "..."}
- Reponse 200 : {"message": "Mot de passe reinitialise."}
- Reponse 400 : {"error": "Lien invalide ou expire."}

Proposition d'implementation :

```
# apps/authentication/views.py

from django.contrib.auth.tokens import default_token_generator
from django.utils.http import urlsafe_base64_encode, urlsafe_base64_decode
from django.utils.encoding import force_bytes
from django.core.mail import send_mail
from django.template.loader import render_to_string
from django.conf import settings

class ForgotPasswordView(APIView):
    """Envoie un email de reset si le compte existe."""
    permission_classes = []
    authentication_classes = []

    def post(self, request):
        email = request.data.get('email', '').strip().lower()
        # Toujours 200 pour ne pas reveler l'existence d'un compte
        msg = {'message': 'Si un compte existe avec cet email, un lien a ete envoye.'}

        if not email:
            return Response(msg)

        try:
            user = User.objects.get(email=email, is_active=True)
        except User.DoesNotExist:
            return Response(msg)
```

```

token = default_token_generator.make_token(user)
uid = urlsafe_base64_encode(force_bytes(user.pk))
reset_url = f'{settings.FRONTEND_URL}/reset-password?uid={uid}&token={token}'

try:
    send_mail(
        subject='Holy Fork – Reinitialisation de mot de passe',
        message=f'Reinitialiser : {reset_url}',
        from_email=settings.DEFAULT_FROM_EMAIL,
        recipient_list=[user.email],
        fail_silently=True,
    )
except Exception:
    pass

return Response(msg)

class ResetPasswordView(APIView):
    """Applique le nouveau mot de passe avec le token reçu par email."""
    permission_classes = []
    authentication_classes = []

    def post(self, request):
        uid = request.data.get('uid')
        token = request.data.get('token')
        new_password = request.data.get('new_password')
        new_password2 = request.data.get('new_password2')

        if not all([uid, token, new_password, new_password2]):
            return Response({'error': 'Tous les champs sont requis.'}, status=400)

        if new_password != new_password2:
            return Response({'new_password': ['Les mots de passe ne correspondent pas.']}), status=400)

        try:
            user_id = urlsafe_base64_decode(uid).decode()
            user = User.objects.get(pk=user_id)
        except (TypeError, ValueError, User.DoesNotExist):
            return Response({'error': 'Lien invalide ou expire.'}, status=400)

        if not default_token_generator.check_token(user, token):
            return Response({'error': 'Lien invalide ou expire.'}, status=400)

        try:
            validate_password(new_password, user)
        except Exception as e:
            return Response({'new_password': list(e.messages)}), status=400)

        user.set_password(new_password)
        user.save()
        return Response({'message': 'Mot de passe reinitialise avec succes.'})

```

URLs a ajouter dans apps/authentication/urls.py :

```

path('change-password/', ChangePasswordView.as_view(), name='change-password'),
path('forgot-password/', ForgotPasswordView.as_view(), name='forgot-password'),
path('reset-password/', ResetPasswordView.as_view(), name='reset-password'),

```

9. Configuration email

Ou : holly_pi/settings.py

Pourquoi : Necessary pour les emails d'invitation (section 10) et le reset password (section 8). En dev, on peut utiliser le backend console (affiche les emails dans le terminal). En prod, un service SMTP type SendGrid.

Proposition :

```

# holly_pi/settings.py – ajouter

import os

# Email
EMAIL_BACKEND = os.environ.get(
    'EMAIL_BACKEND',
    'django.core.mail.backends.console.EmailBackend' # Affiche dans le terminal en dev
)
EMAIL_HOST = os.environ.get('EMAIL_HOST', 'smtp.sendgrid.net')
EMAIL_PORT = int(os.environ.get('EMAIL_PORT', 587))
EMAIL_USE_TLS = True
EMAIL_HOST_USER = os.environ.get('EMAIL_HOST_USER', '')
EMAIL_HOST_PASSWORD = os.environ.get('EMAIL_HOST_PASSWORD', '')

```



```
DEFAULT_FROM_EMAIL = os.environ.get('DEFAULT_FROM_EMAIL', 'Holy Fork <noreply@hollyfork.fr>')
FRONTEND_URL = os.environ.get('FRONTEND_URL', 'http://localhost:5173')
```

En production (.env) :

```
EMAIL_BACKEND=django.core.mail.backends.smtp.EmailBackend
EMAIL_HOST=smtp.sendgrid.net
EMAIL_HOST_USER=apikey
EMAIL_HOST_PASSWORD=SG.xxxxxx
FRONTEND_URL=https://app.hollyfork.fr
```

10. Email d'invitation automatique

Ou : apps/authentication/serializers.py, fin de la methode create()

Pourquoi : Aujourd'hui le gerant doit copier les credentials et les transmettre manuellement (SMS, papier, oral). L'ideal est qu'un email soit envoye automatiquement a l'employe avec ses identifiants.

Important : L'envoi doit etre en fail_silently=True — si l'email echoue, la creation du compte ne doit pas etre annulee. Le gerant a deja les credentials affiches sur son ecran comme backup.

Proposition — ajouter a la fin de create() :

```
def create(self, validated_data):
    # ... (tout le code de creation existant) ...

    # Envoyer un email d'invitation si c'est une creation admin
    if self.context.get('is_admin_invite'):
        self._send_invitation_email(user, validated_data.get('password', ''))

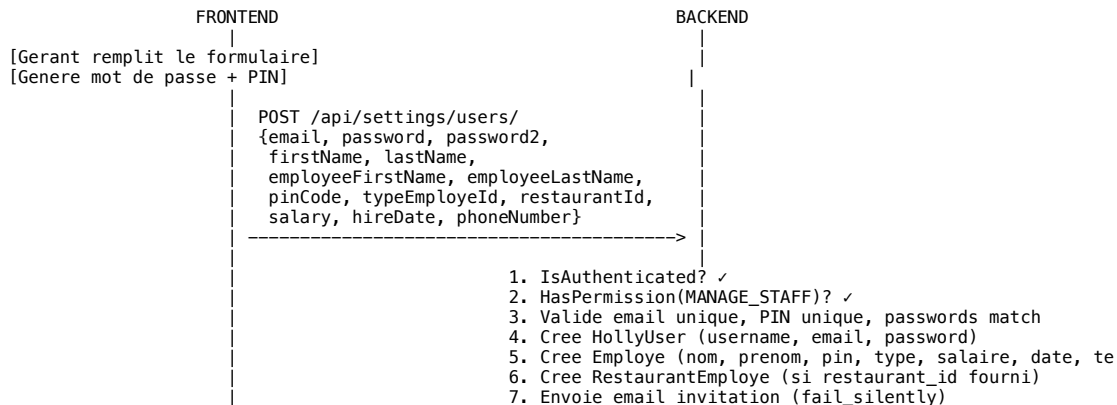
    return user

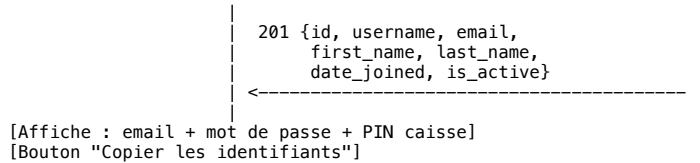
def _send_invitation_email(self, user, password):
    """Envoie un email avec les credentials. Silencieux en cas d'echec."""
    from django.core.mail import send_mail
    from django.conf import settings

    try:
        send_mail(
            subject='Bienvenue sur Holy Fork',
            message=(
                f"Bonjour {user.get_full_name()},\n\n"
                f"Un compte a ete cree pour vous.\n"
                f"Email : {user.email}\n"
                f"Mot de passe : {password}\n\n"
                f"Connectez-vous : {settings.FRONTEND_URL}/login\n"
                f"Pensez a changer votre mot de passe."
            ),
            from_email=settings.DEFAULT_FROM_EMAIL,
            recipient_list=[user.email],
            fail_silently=True,
        )
    except Exception:
        pass
```

Pour un email plus joli en HTML, creer un template dans apps/authentication/templates/emails/invitation.html et utiliser render_to_string + html_message. Mais le texte brut suffit pour commencer.

Resume visuel du flow





Schema de la base de donnees

```
HollyUser (table auth_user standard Django)
| id, username, email, password, first_name, last_name, is_active
|
|— Employe (relation 1:1 via user_id, nullable)
|   | id, user_id, nom, prenom, pin_code, type_employe_id
|   | salaire, date_embauche, numero_telephone
|
|— RestaurantEmploye (table de liaison N:M)
|   | id, restaurant_id, employe_id
|   | UNIQUE (restaurant_id, employe_id)
```

Checklist rapide

- ☐ **P0** — Permission MANAGE_STAFF sur POST /api/settings/users/
- ☐ **P0** — Accepter salary, hire_date, phone_number + restaurant_id optionnel
- ☐ **P1** — Filtrer GET /api/settings/users/ par restaurant
- ☐ **P1** — PATCH /api/settings/users/{id}/ avec is_active
- ☐ **P1** — Ajouter email dans GET /api/employees/
- ☐ **P1** — Validation hierarchique sur changement de role
- ☐ **P2** — POST /api/auth/change-password/
- ☐ **P2** — POST /api/auth/forgot-password/ + reset-password/
- ☐ **P2** — Config EMAIL_BACKEND dans settings.py
- ☐ **P2** — Email automatique a l'invitation

Temps total estime : environ 2h.